

CSV File Handling

08 April 2026 06:35

CSV stands for Comma-Separated Values. A CSV file is a plain text file used to store tabular data such as:

- Students record
- Employee details
- Sales report
- Product lists
- Marksheets

Each line in a CSV file represents One row of data. Values in a row are separated by commas (by default).

heading →

records →

R.no	sub1	sub2	sub3	sub4
1	80	83	85	77
2	76	55	66	81
3	90	94	87	66
4	100	94	95	91
5	89	97	98	99

Tabular representation of data.

Example of CSV file:

Roll, Name, sub1, sub2, sub3

1, Vaanya, 90, 91, 92
2, Kapish, 88, 89, 90
3, Aarav, 79, 88, 87
4, Jiya, 100, 99, 98
5, Aadya, 90, 99, 100

Roll, Name, sub1, sub2, and sub3 are column names
Each next line is a record.

Why CSV Files are used?

CSV files are widely used because:

- They are simple and easy to understand.
- They can be opened in spreadsheet software like Excel.
- They are smaller than database files.
- They are easy to transfer between systems.
- They are supported by many programming languages.

Working with CSV files in Python

Python provides a built in module called `csv` for handling CSV files.

`import csv`

The `csv` module helps us:

- Read CSV files.
- Write CSV files.
- Append data to CSV files.
- Read rows as dictionary or list.
- Handling delimiters.

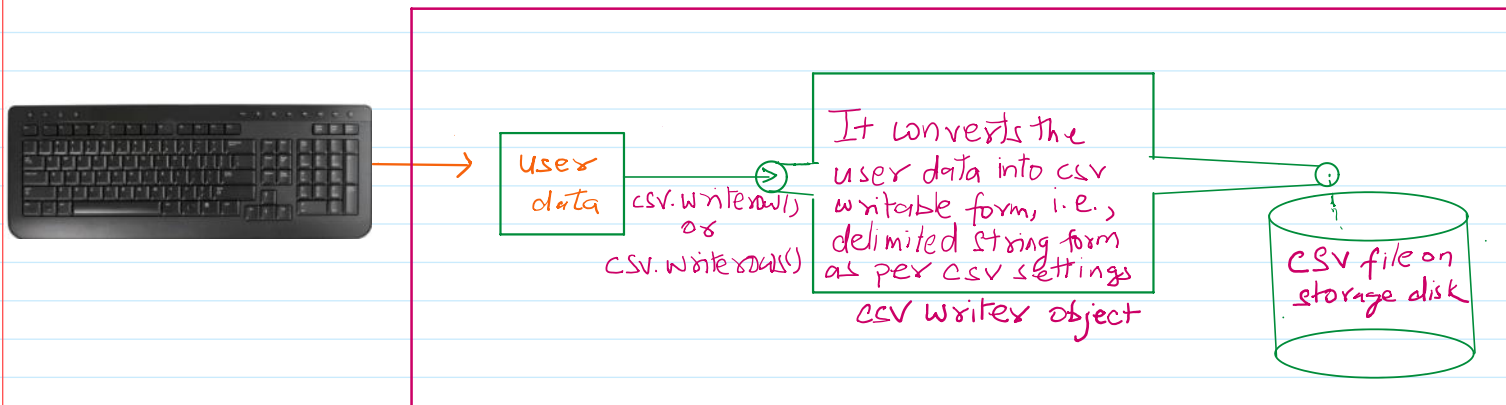
Writing in CSV Files

The writing into CSV files involves the conversion of the user data into the writable delimited form and then storing it in the form of CSV file. For writing onto a CSV file you normally use the following three key player functions.

These are:

<code>csv.writer()</code>	Returns a writer object which writes data into CSV files.
<code><writerobject>.writerow()</code>	Writes one row of data onto the writer object.
<code><writerobject>.writerows()</code>	Writes multiple rows of data onto the writer object.

Since CSV files are delimited flat files and before writing onto them, the data must be in **CSV writable delimited form**, it is important to convert the received user data into the form appropriate for the CSV files. **This task is performed by the writer object.** The data row written. To a writer object. (Written using `writerow()` or `writerows()` functions) Gets converted to CSV. A writable delimited form and then written on to the linked CSV file on the disk.



Role of the CSV writer object.

In order to write into a CSV file. You need to do the following:

1. Import `csv` module.
2. Open CSV file in a file handle. (Just as you open other text files) e.g.,
with `open("student.csv", "w")` as `fo`:
3. Create the **writer object** by using the syntax shown below:

```
<name_of_writer_object> = csv.writer(<file-handle>, [delimiter = <delimiter_character>])
```

```
stuWriter = csv.writer(fo)
```

Or

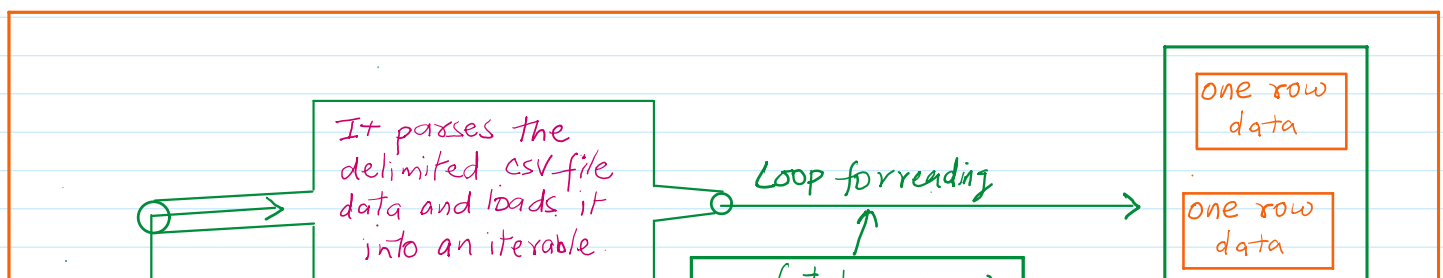
```
stuWriter = csv.writer(fo, delimiter = '#')
```
4. Obtain user data and form a Python sequence (list or tuple) out of it. e.g.

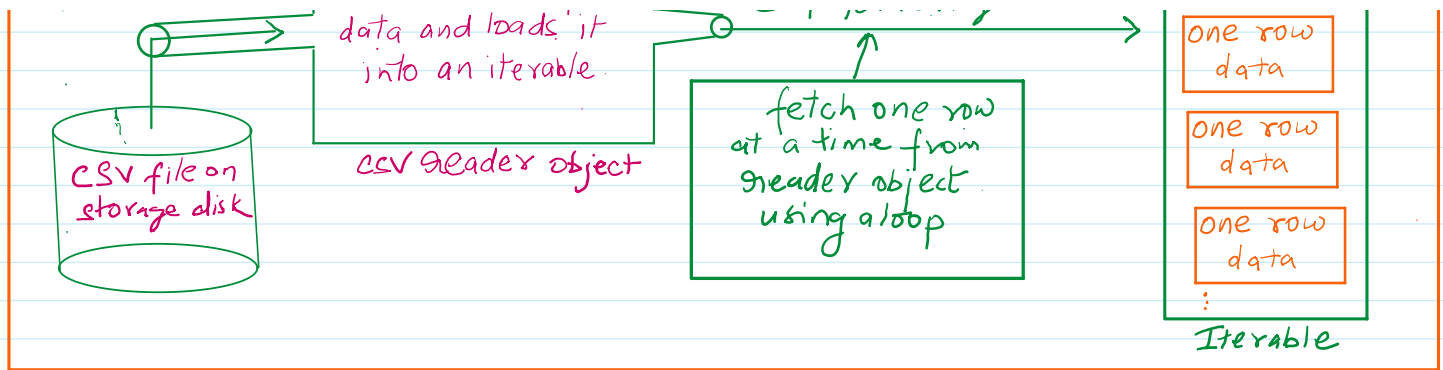
```
stuRec = (11, "Max", 79.0, 81.2, 83.5)
```
5. Write the Python sequence containing the user data onto the writer object using `csv.writerow()` or `csv.writerows()` functions.

```
csv.writerow(stuRec)
```

6. Once the above is done, it automatically closes the file.

Reading in CSV file





Role of csv.reader object

Reading from a CSV file involves loading of a CSV file's data **parsing** it. (i.e., removing its delimiters) Loading it in a Python iterable and then reading from this iterable. Recall that. Any Python sequence that can be iterated over in a for loop is a Python iterable, example, list, tuples, and strings are all Python iterables.

Let us now learn to read from CSV files. In order to read from a CSV file you need to do the following:

1. Import csv module.
2. Open CSV file in a file handle in read mode. (just as you open other text files)
`with open("filename.csv", "r") as fo:`

The file being opened must already exist, otherwise an exception will get raised. Your code should be able to handle the exception, that is through **try...except**.

3. Create the reader object by using the syntax as shown below:
`<name_of_reader_object> = csv.reader(<file_handle>,
[delimiter=<delimiter_character>])`

Example:

```
stuReader = csv.reader(fo)
Or
stuReader = csv.reader(fo, delimiter='#')
```

4. The reader object stores the past data in the form of iterable. And thus you can fetch from it row by row through a traditional for loop. One row at a time.

```
for rec in stuReader:
    print(rec) # Or do any other processing.
```

5. The file is automatically closed as we are using **with open()** syntax.

Code to write or append data in the csv file:

```
import csv
# Write a program to create a CSV file to store student data (The roll number,
Name, Marks)
```

```
# Obtain data from the user and write 5 records into the file.
fileName = input("Enter your csv file name: ")
with open("CSVFileHandling/CSVFiles/"+fileName+".csv", "a", newline='') as fo:
    num_of_records = 5
    li=[]
    stu_writer = csv.writer(fo)
    stu_writer.writerow(['Roll_num','Name', 'Marks'])
    for i in range(num_of_records):
        roll_num = int(input("Enter the roll number: "))
        name = input("Enter name of the student: ")
        marks = float(input("Enter the average marks: "))
        li.append([roll_num, name, marks])
    stu_writer.writerows(li)
```

Output:

```
CSVFileHandling > CSVFiles > class6.csv
1 Roll_num,Name,Marks
2 1,Vaanya,93.6
3 2,Jiya,91.2
4 3,Gargi,87.8
5 4,Amy,77.8
6 5,Sanvi,80.7
```

Code to read data from csv files

```
import csv
file_name = input("Enter your file name: ")
with open("CSVFileHandling/CSVFiles/"+file_name+".csv", "r") as fh:
    csvrecords = csv.reader(fh)
    for rec in csvrecords:
        print(rec)
```

Output:

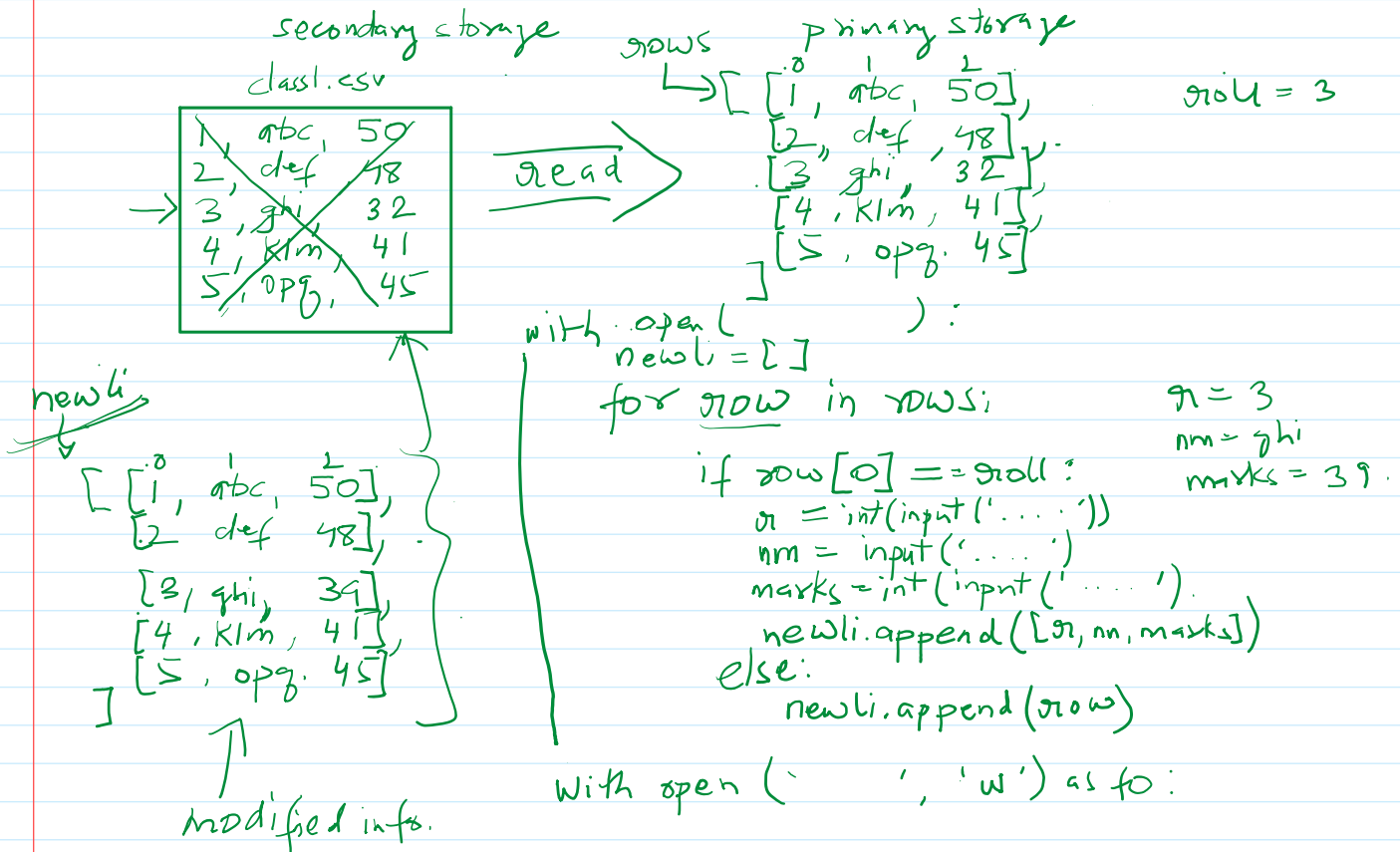
```
Enter your file name: class6
['Roll_num', 'Name', 'Marks']
['1', 'Vaanya', '93.6']
['2', 'Jiya', '91.2']
['3', 'Gargi', '87.8']
['4', 'Amy', '77.8']
['5', 'Sanvi', '80.7']
```

Homework

1. Write a Python program to create a Student Management System using CSV files. The program should allow the user to:
 - Add new student records
 - Display all student records
 - Search a student by roll number
 - Update marks of a student
 - Delete a student record

- Calculate total marks and percentage
 - Handle exceptions such as invalid input, missing file, and incorrect roll number
2. Write a Python program to manage employee records using CSV files. Each employee record should contain employee ID, name, department, salary, and bonus. The program should:
 - Add employee details to a CSV file
 - Display all employee records
 - Search for an employee using employee ID
 - Update salary and bonus
 - Delete an employee record
 - Calculate annual salary
 - Use DictReader() and DictWriter() for reading and writing records
 3. Write a Python program for a Library Management System using CSV files. The program should maintain book details such as book ID, title, author, price, and quantity. It should:
 - Add new books
 - Search for books by title or author
 - Update book quantity after issue or return
 - Delete a book record
 - Display all available books
 - Handle file-related exceptions and invalid user inputs
 4. Write a Python program to create a Sales Report System using CSV files. The CSV file should store product ID, product name, quantity sold, price per unit, and total sale amount. The program should:
 - Add new sales records
 - Display all sales records
 - Search products by product ID
 - Update product price or quantity
 - Calculate total sales for all products
 - Display the product with the highest sale amount
 - Use exception handling to manage errors
 5. Write a Python program to create an Examination Result System using CSV files. The program should store student name, roll number, subject marks, total marks, percentage, and grade. The program should:
 - Accept details of multiple students
 - Store records in a CSV file
 - Read and display all records
 - Search for a student using roll number
 - Update marks of a student
 - Automatically calculate percentage and grade
 - Handle exceptions such as missing files, invalid marks, and invalid roll numbers
1. Write a Python program to create a Student Management System using CSV files. The program should allow the user to:
 - Add new student records
 - Display all student records
 - Search a student by roll number
 - Update marks of a student
 - Delete a student record
 - Calculate total marks and percentage

- Handle exceptions such as invalid input, missing file, and incorrect roll number



To store and retrieve data in the form of a dictionary from CSV files in Python. You mainly use the built in CSV module, especially:

- `csv.DictWriter` -> For storing (writing) dictionaries.
- `csv.DictReader` -> for retrieving (reading) dictionaries.

CSV file store tabular data where each row is a record and columns are separated by commas.

CSV as Dictionary

- First row = Keys (column names)
- Each next row = dictionary (key->value)

Example CSV:

```
roll,name,marks
1,Shiv,90
2,Ram,91
```

Python:

```
{'roll':1, 'name':'Shiv', 'marks':90}
{'roll':2, 'name':'Ram', 'marks':91}
```

Python automatically maps columns to dictionary keys using DictReader.

Writing Dictionary to CSV(Store Data)

```
import csv
# List of dictionaries
students = [
    {'roll':1, 'name':'Shiv', 'marks':90},
    {'roll':2, 'name':'Ram', 'marks':94},
    {'roll':3, 'name':'Kartik', 'marks':88},
    {'roll':4, 'name':'Lakshmi', 'marks':92},
    {'roll':5, 'name':'Ganesh', 'marks':93},
]
# Field names (CSV columns)
fields = ['roll', 'name', 'marks']
# Writing to CSV.
with open("CSVFiles/students.csv", "w", newline='') as fo:
    writer = csv.DictWriter(fo, fieldnames=fields)

    writer.writeheader() # Write column names
    writer.writerows(students) # Writing data
```

maps dictionary keys → CSV columns

writes the column names.

writes multiple records.

Output:

```
CSVFiles > students.csv
1 roll,name,marks
2 1,Shiv,90
3 2,Ram,94
4 3,Kartik,88
5 4,Lakshmi,92
6 5,Ganesh,93
7
```

Reading CSV as dictionary (retrieve data)

```
import csv
with open("CSVFiles/students.csv", "r") as fo:
    reader = csv.DictReader(fo)

    for row in reader:
        print(row)
```

Output:

```
{'roll': '1', 'name': 'Shiv', 'marks': '90'}
{'roll': '2', 'name': 'Ram', 'marks': '94'}
{'roll': '3', 'name': 'Kartik', 'marks': '88'}
{'roll': '4', 'name': 'Lakshmi', 'marks': '92'}
{'roll': '5', 'name': 'Ganesh', 'marks': '93'}
```

Each row becomes a dictionary.
Keys come from header row.
Values are strings by default.

Searching in csv file:

```
import csv
with open("CSVFiles/students.csv", "r") as fo:
    reader = csv.DictReader(fo)
    rollNumb = int(input('Enter roll number to search: '))
    for row in reader:
        if int(row['roll']) == rollNumb:
            print(row)
            break
    else:
        print(f'Roll number {rollNumb} is not present')
```

Output:

```
Enter roll number to search: 3
{'roll': '3', 'name': 'Kartik', 'marks': '88'}
```

Another execution:

```
Enter roll number to search: 6
Roll number 6 is not present
```

Deleting record from the csv files containing dictionary

```
import csv
def delete_record(rollNumb:int):
    data = []
    found = False
    with open("CSVFiles/students.csv", "r") as fo:
        reader = csv.DictReader(fo)
        for row in reader:
            if int(row['roll']) != rollNumb:
                data.append(row) # Keep the record
            else:
                found = True # mark found

    if found:
        print('Record Deleted successfully')
        with open('CSVFiles/students.csv', 'w', newline='') as fo:
            fields = ['roll', 'name', 'marks']
            writer = csv.DictWriter(fo, fieldnames=fields)
            writer.writeheader()
            writer.writerows(data)
    else:
        print(f'Roll number {rollNumb} is not present')
if __name__ == '__main__':
    rollNumb = int(input("Enter roll number to delete the record of student: "))
    delete_record(rollNumb)
```

Before execution:


```

CSVFiles > students.csv
1 roll,name,marks
2 1,Shiv,90
3 2,Ram,94
4 3,Kartik,88
5 4,Lakshmi,92
6 5,Ganesh,93
7

```

Output:

```

Enter roll number to delete the record of student: 4
Record Deleted successfully

```

Another execution:

```

Enter roll number to delete the record of student: 3
Roll number 3 is not present

```

```

CSVFiles > students.csv
1 roll,name,marks
2 1,Shiv,90
3 2,Ram,94
4 5,Ganesh,93
5

```

```

import csv
def append_student(record:dict):
    # Field names (CSV columns)
    fields = ['roll', 'name', 'marks']
    # Writing to CSV.
    try:
        with open("CSVFiles/students.csv", "a", newline='') as fo:
            writer = csv.DictWriter(fo, fieldnames=fields)
            writer.writerow(record) # Appended dictionary
            print('Record appended successfully')
    except FileNotFoundError:
        print('File not found')

if __name__ == '__main__':
    di = dict()
    rn = int(input('Enter roll number: '))
    nm = input('Enter name of student: ')
    marks = int(input('Enter marks: '))
    di['roll'] = rn
    di['name'] = nm
    di['marks'] = marks
    append_student(di)

```

Output:

```
Enter roll number: 8
Enter name of student: Harish
Enter marks: 99
Record appended successfully
```

CSVFiles > students.csv		
1	roll,name,marks	
2	1,Shiv,90	
3	2,Ram,94	
4	3,Kartik,88	
5	4,Lakshmi,92	
6	5,Ganesh,93	
7	6,Karan,94	
8	7,Sam,89	
9	8,Harish,99	

Homework:

Solve same questions given above using dictionary in Python

Project:

Write a Python program to create a Student Management System using CSV files. The program should allow the user to:

- Add new student records
 - On existing file
 - Create new csv file
- Display all student records
 - Enter your file name
- Search a student by roll number
 - Enter file name
 - Enter the roll number of the student in that class
- Update marks of a student
 - Enter file name (file must exist)
 - Update all the students unit_test marks
- Delete a student record
 - Enter the file name
 - Enter roll number to delete the record
- Calculate total marks and percentage
 - Enter file name and do the calculation based on unit_test columns
- Handle exceptions such as invalid input, missing file, and incorrect roll number